恒生指数情绪因子研究

量化研究报告

作者：姓名 **日期**：2025 **数据**：恒生指数（2022–2024），预计 n=747 个交易日

执行摘要

本研究探讨社交媒体情绪投票数据（看多/看空票数）是否能预测恒生指数次日收益。 我们构建三种交易策略，分别在含手续费与不含手续费两种情景下评估其表现。

核心结论：

- 信号质量统计详见第 5 节。
- 各策略绩效对比详见第 8 节和第 10 节。
- 手续费影响已直接量化于回测指标表中。

In [1]:
```
from pathlib import Path

import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import Markdown, display
from scipy import stats
from scipy.stats import mstats, ttest_1samp

warnings.filterwarnings("ignore")

plt.rcParams["figure.dpi"] = 120
plt.rcParams["font.size"] = 11
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.alpha"] = 0.3
sns.set_style("whitegrid")

print("Libraries loaded successfully")
print(f"Pandas: {pd.__version__}, NumPy: {np.__version__}")
```

```
Libraries loaded successfully
Pandas: 2.3.3, NumPy: 2.3.4
```

1. 研究背景

本报告评估日度社交媒体情绪投票能否作为恒生指数的短期 Alpha 信号。 核心假设是：t 日的净看多情绪包含对次日指数表现有预测价值的信息。

研究分三个层次展开：

1. 数据验证与特征构建。
2. 信号诊断，包括 IC、ICIR 及信号衰减分析。
3. 策略设计、回测与稳健性检验。

本研究目标侧重实用性：判断情绪信号的强度、稳定性和可交易性是否足以支持一套简单的系统化策略。

In [2]:
```
DATA_FILE = Path("HSI.xlsx")
VOTE_TOLERANCE = 1e-6
SUMMARY_COLUMNS = ["Open", "High", "Low", "Close", "Up votes", "Down votes"]

if not DATA_FILE.exists():
    raise FileNotFoundError(
        f"Input file not found: {DATA_FILE.resolve()}. "
        "Place HSI.xlsx in the notebook working directory before execution."
    )

df_raw = pd.read_excel(DATA_FILE)
df_raw["Date"] = pd.to_datetime(df_raw["Date"])
df_raw = df_raw.sort_values("Date").reset_index(drop=True)

required_columns = {"Date", "Open", "High", "Low", "Close", "Up votes", "Down votes
missing_columns = required_columns.difference(df_raw.columns)
if missing_columns:
    raise ValueError(f"Missing required columns: {sorted(missing_columns)}")

inspection = pd.DataFrame(
    {
        "dtype": df_raw.dtypes.astype(str),
        "missing": df_raw.isna().sum(),
    }
)

duplicate_rows = int(df_raw.duplicated().sum())
df_raw["vote_sum"] = df_raw["Up votes"] + df_raw["Down votes"]
df_raw["vote_anomaly"] = (df_raw["vote_sum"] - 1).abs() > VOTE_TOLERANCE
vote_anomalies = int(df_raw["vote_anomaly"].sum())

df_raw["Up votes"] = df_raw["Up votes"].ffill()
df_raw["Down votes"] = df_raw["Down votes"].ffill()

summary_stats = df_raw[SUMMARY_COLUMNS].describe().T

print(f"Shape: {df_raw.shape}")
print(f"Date range: {df_raw['Date'].min().date()} to {df_raw['Date'].max().date()}"
print(f"Duplicate rows: {duplicate_rows}")
print(f"Vote sum anomalies: {vote_anomalies}")
display(df_raw.head())
display(inspection)
display(summary_stats)
```

```
Shape: (747, 9)
Date range: 2022-02-23 to 2025-03-12
Duplicate rows: 0
Vote sum anomalies: 4
```

	Date	Open	High	Low	Close	Up votes	Down votes	vote_sum	vote_anomal
0	2022-02-23	22088.79	22423.51	21983.10	22154.08	0.28	0.72	1.0	Fals
1	2022-02-24	23268.03	23376.37	22786.39	22901.56	0.69	0.31	1.0	Fals
2	2022-02-25	22908.10	23046.86	22718.15	22767.18	0.36	0.64	1.0	Fals
3	2022-02-28	22818.17	22818.17	22406.12	22713.02	0.61	0.49	1.1	Tru
4	2022-03-01	22739.84	22937.56	22591.73	22761.71	0.49	0.51	1.0	Fals

	dtype	missing
Date	datetime64[ns]	0
Open	float64	0
High	float64	0
Low	float64	0
Close	float64	0
Up votes	float64	195
Down votes	float64	195

	count	mean	std	min	25%	50%	75%	
Open	747.0	19050.836104	1883.937812	14830.69	17574.215	19254.38	20359.595	24
High	747.0	19228.443320	1903.573581	15113.15	17689.605	19378.90	20550.935	24
Low	747.0	18870.628969	1853.857710	14597.31	17400.990	19072.72	20141.360	24
Close	747.0	19049.844980	1888.092373	14687.02	17524.560	19229.97	20359.555	24
Up votes	747.0	0.543266	0.099264	0.24	0.470	0.55	0.610	
Down votes	747.0	0.456734	0.098628	0.18	0.390	0.45	0.530	

2. 数据概览

数据集包含恒生指数每日价格及两个情绪投票字段：`Up votes`（看多票数比例）和 `Down votes`（看空票数比例）。 初步验证重点关注时间顺序、缺失值处理、重复行检测，以及情绪份额之和是否近似为 1。

对缺失情绪数据进行前向填充后，数据集可进入特征工程阶段。 上方描述统计提供了价格水平与情绪输入截面的基准视图。

In [3]:
```
df = df_raw.copy()

df["return_open"] = df["Open"].shift(-1) / df["Open"] - 1
df["return_close"] = df["Close"].pct_change()

df["sentiment_score"] = df["Up votes"] - df["Down votes"]
df["vote_imbalance"] = df["sentiment_score"].abs()

q80 = df["sentiment_score"].quantile(0.8)
q20 = df["sentiment_score"].quantile(0.2)
df["extreme_bull"] = df["sentiment_score"] >= q80
df["extreme_bear"] = df["sentiment_score"] <= q20

df["sentiment_lag1"] = df["sentiment_score"].shift(1)
df["sentiment_lag2"] = df["sentiment_score"].shift(2)
df["sentiment_lag3"] = df["sentiment_score"].shift(3)

ranked_sentiment = df["sentiment_score"].rank(method="first")
df["sentiment_quantile"] = pd.qcut(
    ranked_sentiment,
    q=5,
    labels=["Q1", "Q2", "Q3", "Q4", "Q5"],
)

feature_summary = df[
    ["sentiment_score", "vote_imbalance", "return_open", "return_close"]
].describe().T
display(feature_summary)
```

	count	mean	std	min	25%	50%	75%
sentiment_score	747.0	0.086533	0.197689	-0.520000	-0.060000	0.100000	0.220000
vote_imbalance	747.0	0.174833	0.126379	0.000000	0.060000	0.160000	0.260000
return_open	746.0	0.000255	0.017485	-0.068238	-0.010621	-0.000868	0.010098
return_close	746.0	0.000232	0.017215	-0.094070	-0.010544	-0.000686	0.009069

3. 特征工程

核心预测变量为 `sentiment_score = Up votes - Down votes`，衡量市场情绪的净方向。 同时构建 `vote_imbalance = |sentiment_score|` 作为情绪强度（共识程度）的代理指标。

为检验不同经济含义，本报告构建以下特征：

- `return_open`：次日开盘对开盘收益率，与 t 日已知信号对齐。
- `extreme_bull` 和 `extreme_bear`：情绪极值的尾部状态标记。
- `sentiment_lag1` 至 `sentiment_lag3`：滞后信号，用于衰减分析。
- `sentiment_quantile`：五分位情绪分组，用于单调性检验。

In [4]:
```
diagnostics = pd.DataFrame(
    {
        "metric": [
            "Extreme bull days",
            "Extreme bear days",
            "Median vote imbalance",
            "Valid next-day returns",
        ],
        "value": [
            int(df["extreme_bull"].sum()),
            int(df["extreme_bear"].sum()),
            float(df["vote_imbalance"].median()),
            int(df["return_open"].notna().sum()),
        ],
    }
)

quantile_dist = df["sentiment_quantile"].value_counts(sort=False, dropna=False)
display(diagnostics)
display(quantile_dist.to_frame("count"))
```

	metric	value
0	Extreme bull days	151.00
1	Extreme bear days	155.00
2	Median vote imbalance	0.16
3	Valid next-day returns	746.00

sentiment_quantile	count
Q1	150
Q2	149
Q3	149
Q4	149
Q5	150

4. 探索性数据分析

图 1. 样本区间内恒生指数收盘价时序。

In [5]:
```
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(df["Date"], df["Close"], color="steelblue", linewidth=1.2)
ax.set_title("HSI Close Price")
ax.set_xlabel("Date")
ax.set_ylabel("Close")
plt.show()
```

图 2. 价格与情绪得分双轴图，展示两者随时间的联动关系。

In [6]:
```
fig, ax1 = plt.subplots(figsize=(14, 5))
ax2 = ax1.twinx()
line1 = ax1.plot(df["Date"], df["Close"], color="steelblue", alpha=0.75, label="Clo
line2 = ax2.plot(df["Date"], df["sentiment_score"], color="darkorange", alpha=0.75,
ax1.set_title("HSI Price vs Sentiment Score")
ax1.set_xlabel("Date")
ax1.set_ylabel("Close", color="steelblue")
ax2.set_ylabel("Sentiment Score", color="darkorange")
lines = line1 + line2
ax1.legend(lines, [line.get_label() for line in lines], loc="upper left")
plt.show()
```

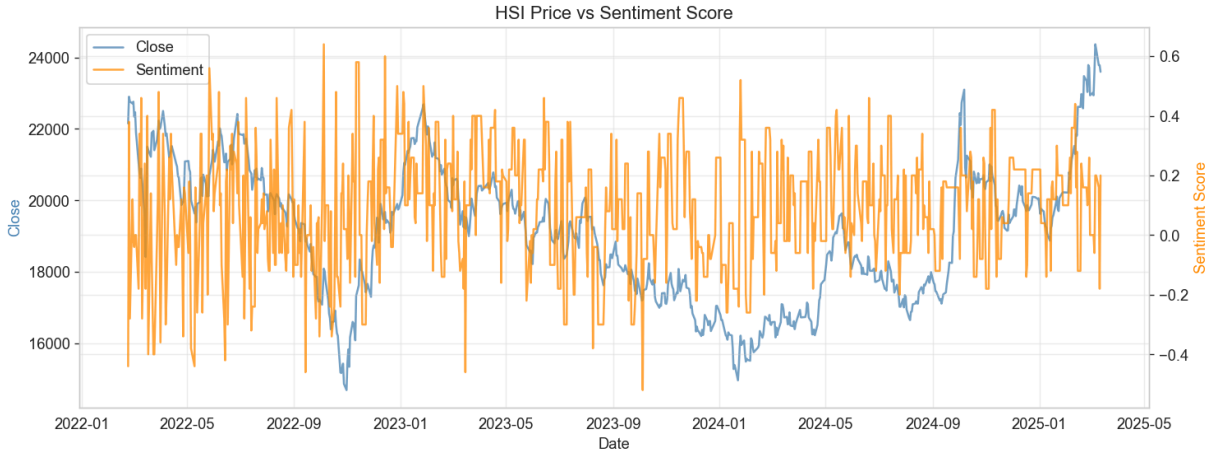


图 3. 情绪得分分布图，用于评估其对称性、偏度及在中性附近的集中程度。

In [7]:
```
fig, ax = plt.subplots(figsize=(10, 5))
sentiment = df["sentiment_score"].dropna()
```

```
ax.hist(sentiment, bins=50, density=True, alpha=0.6, color="steelblue", label="Hist
sentiment.plot.kde(ax=ax, color="darkorange", linewidth=2, label="KDE")
ax.set_title("Sentiment Score Distribution")
ax.set_xlabel("Sentiment Score")
ax.set_ylabel("Density")
ax.legend()
plt.show()
```

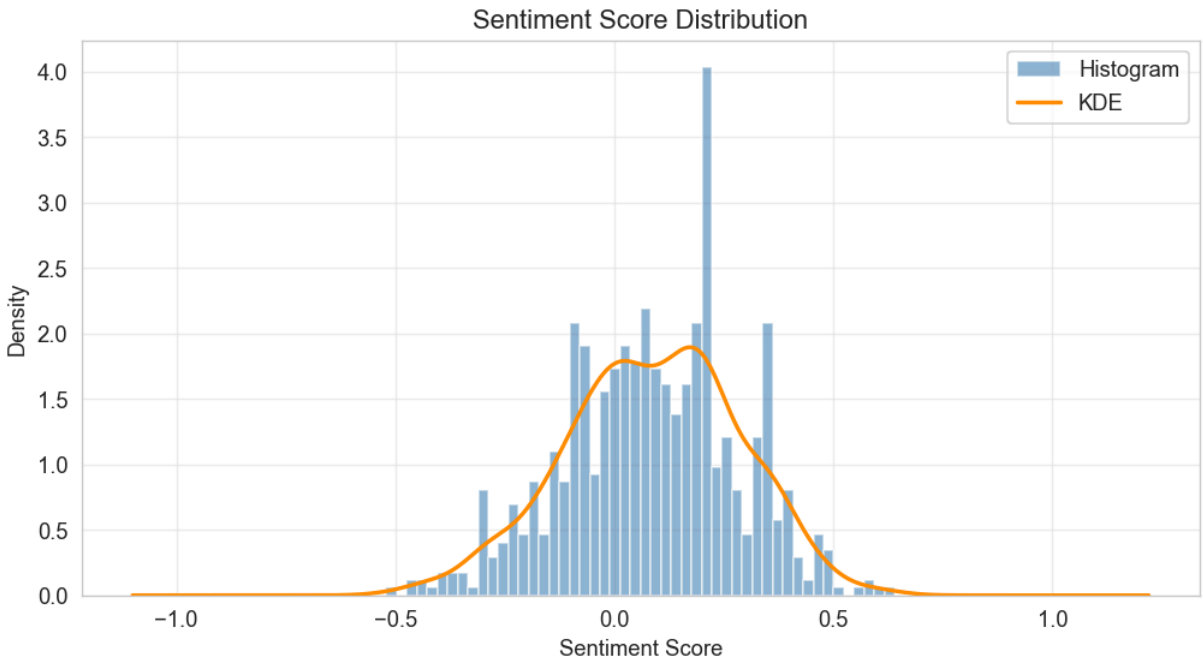


图 4. 情绪得分与次日收益率的散点图及线性趋势拟合。

In [8]:
```
scatter_df = df[["sentiment_score", "return_open"]].dropna()
slope, intercept, r_value, p_value, _ = stats.linregress(
    scatter_df["sentiment_score"],
    scatter_df["return_open"],
)

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(scatter_df["sentiment_score"], scatter_df["return_open"], alpha=0.3, s=1
x_line = np.linspace(scatter_df["sentiment_score"].min(), scatter_df["sentiment_sco
ax.plot(x_line, slope * x_line + intercept, color="crimson", linewidth=2)
ax.set_title(f"Sentiment vs Next-Day Return (r={r_value:.3f}, p={p_value:.3f})")
ax.set_xlabel("Sentiment Score")
ax.set_ylabel("Next-Day Open Return")
plt.show()
```

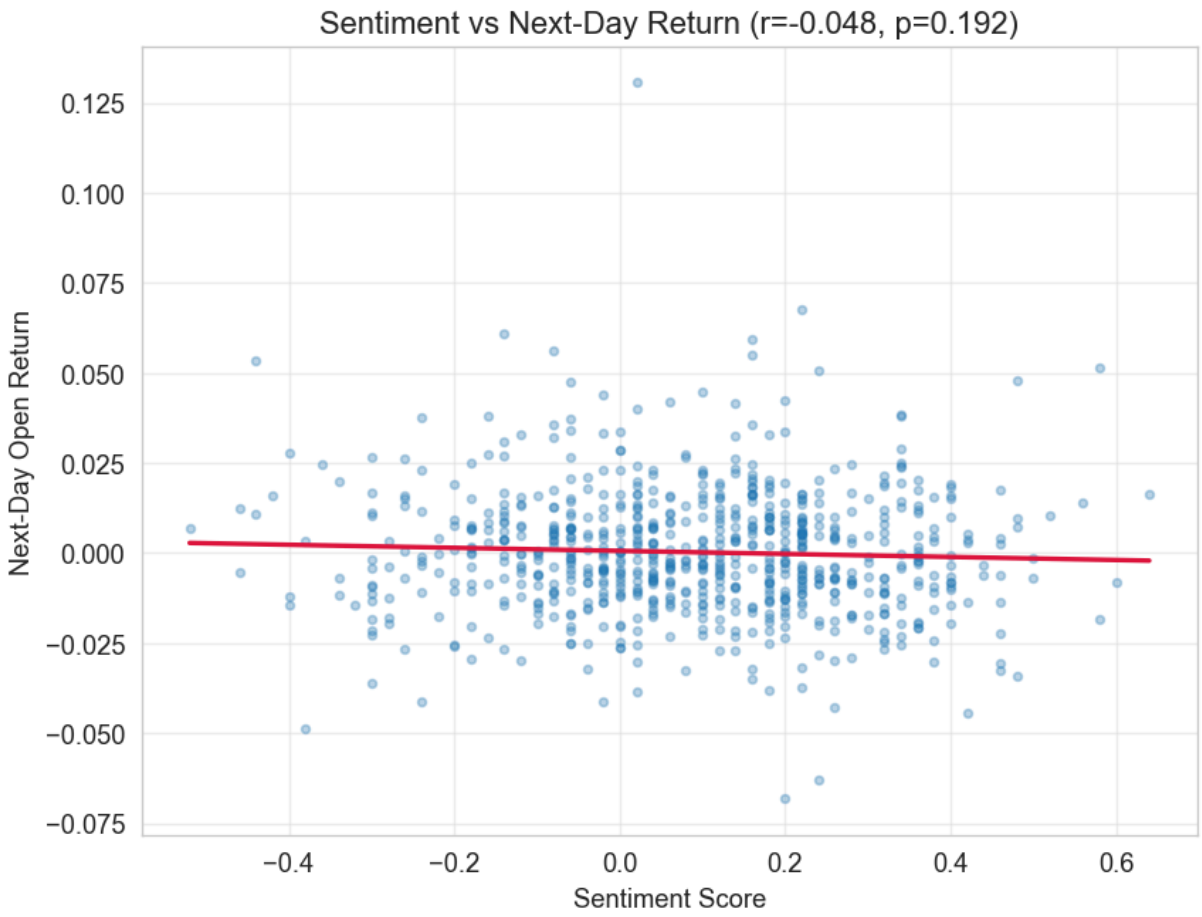


图 5. 各情绪分位组的次日平均收益率，用于检验单调性。

In [9]:
```
quantile_returns = (
    df.groupby("sentiment_quantile", observed=False)["return_open"]
    .mean()
    .reindex(["Q1", "Q2", "Q3", "Q4", "Q5"])
)

fig, ax = plt.subplots(figsize=(8, 5))
quantile_returns.plot(
    kind="bar",
    ax=ax,
    color=["#d73027", "#fc8d59", "#fee090", "#91bfdb", "#4575b4"],
)
ax.axhline(0, color="black", linestyle="--", alpha=0.6)
ax.set_title("Average Next-Day Return by Sentiment Quantile")
ax.set_xlabel("Sentiment Quantile")
ax.set_ylabel("Mean Return")
ax.tick_params(axis="x", rotation=0)
plt.show()
```

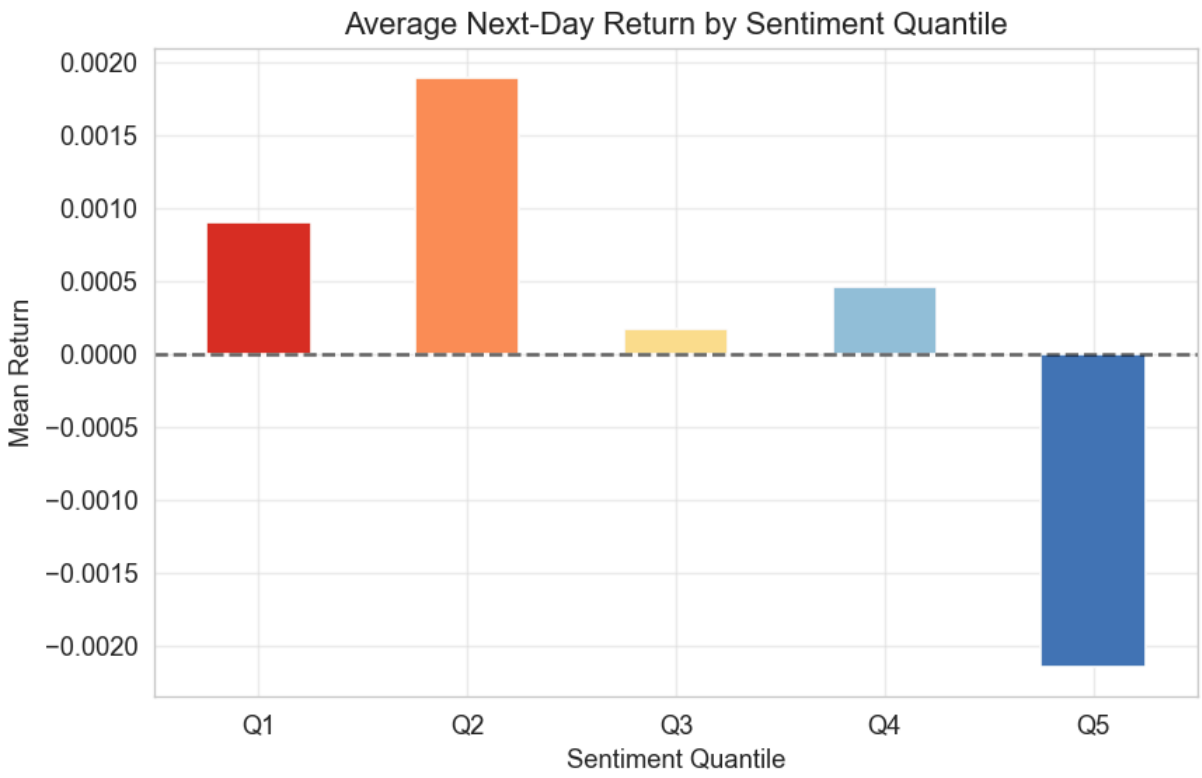


图 6. 各情绪分组的滚动扩展平均收益率，展示信号的时间持续性。

In [10]:
```
fig, ax = plt.subplots(figsize=(14, 6))
colors = ["#d73027", "#fc8d59", "#fee090", "#91bfdb", "#4575b4"]
for quantile, color in zip(["Q1", "Q2", "Q3", "Q4", "Q5"], colors):
    subset = df.loc[df["sentiment_quantile"] == quantile, ["Date", "return_open"]].
    if subset.empty:
        continue
    ax.plot(
        subset["Date"].values,
        subset["return_open"].expanding().mean().values,
        label=quantile,
        color=color,
        linewidth=2,
    )
ax.set_title("Expanding Average Return by Sentiment Quantile")
ax.set_xlabel("Date")
ax.set_ylabel("Expanding Mean Return")
ax.legend()
plt.show()
```

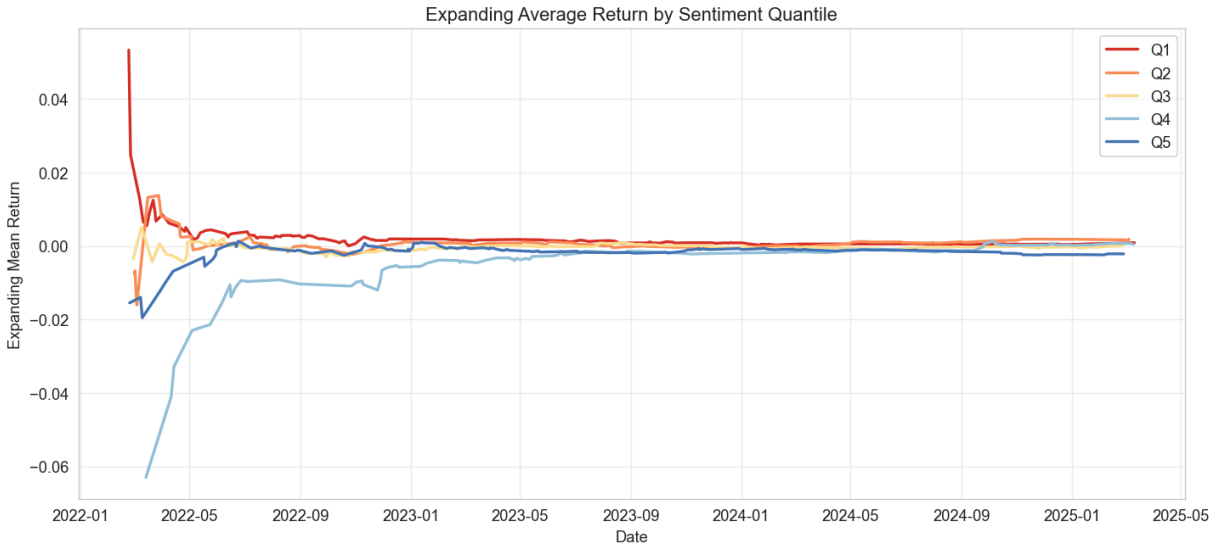
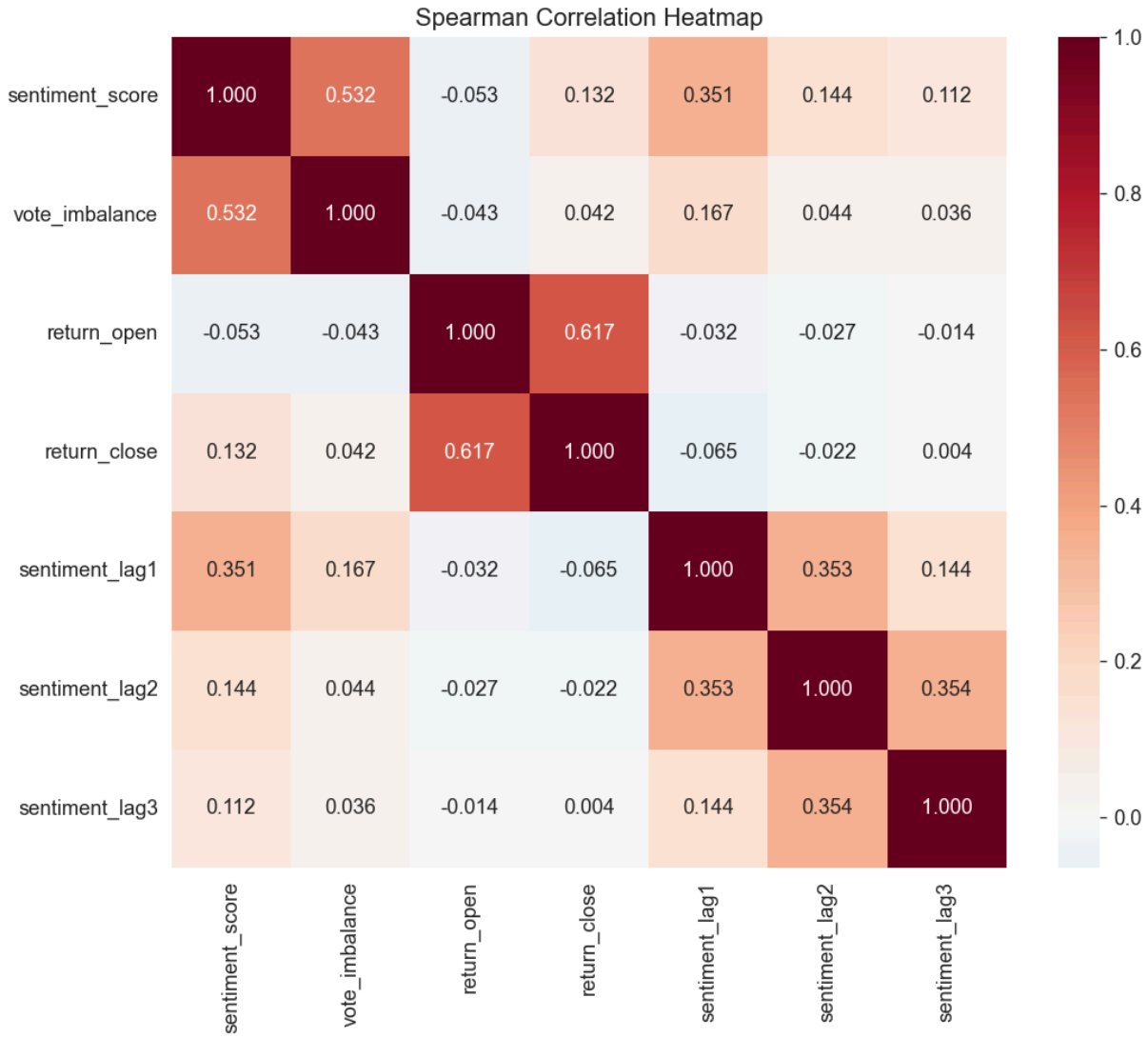


图 7. 收益率、情绪及滞后情绪特征之间的 Spearman 相关系数热力图。

In [11]:
```
corr_cols = [
    "sentiment_score",
    "vote_imbalance",
    "return_open",
    "return_close",
    "sentiment_lag1",
    "sentiment_lag2",
    "sentiment_lag3",
]
corr = df[corr_cols].corr(method="spearman")

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt=".3f", cmap="RdBu_r", center=0, ax=ax)
ax.set_title("Spearman Correlation Heatmap")
plt.show()
```



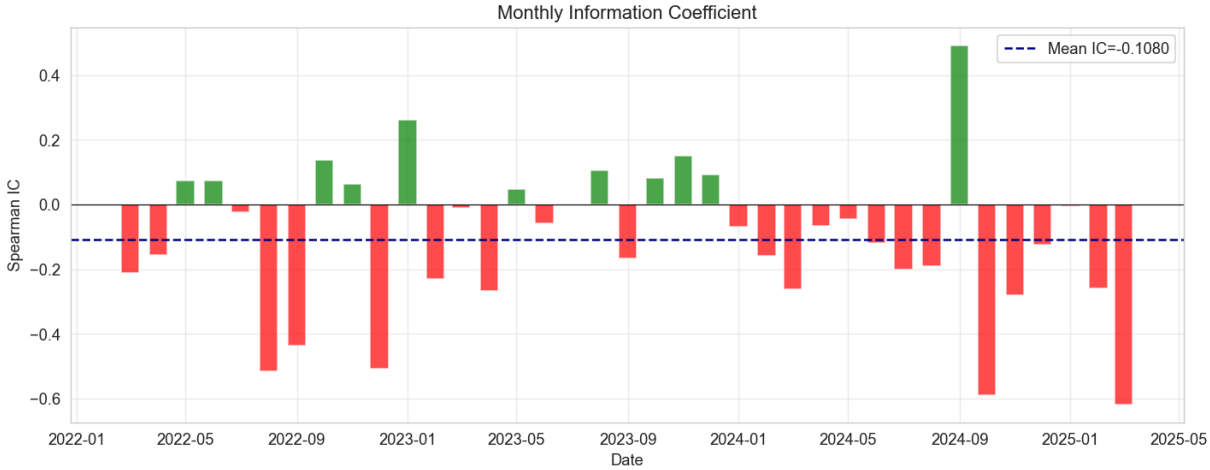
5. 信号分析

本节评估情绪是否具有预测信号价值。 主要质量指标为 `sentiment_score_t` 与 `return_open_t+1` 之间的 Spearman 信息系数（IC）。 同时考察 IC 时序的稳定性、对零的 t 检验，以及随预测期延长信号强度的衰减情况。

In [12]:
```
MIN_MONTHLY_OBSERVATIONS = 5

working = df.copy()
working["year_month"] = working["Date"].dt.to_period("M")
monthly_ic_records = []

for period, group in working.groupby("year_month"):
    clean = group[["sentiment_score", "return_open"]].dropna()
    if len(clean) < MIN_MONTHLY_OBSERVATIONS:
        continue
    ic, p_val = stats.spearmanr(clean["sentiment_score"], clean["return_open"])
    monthly_ic_records.append(
        {
            "period": period,
```

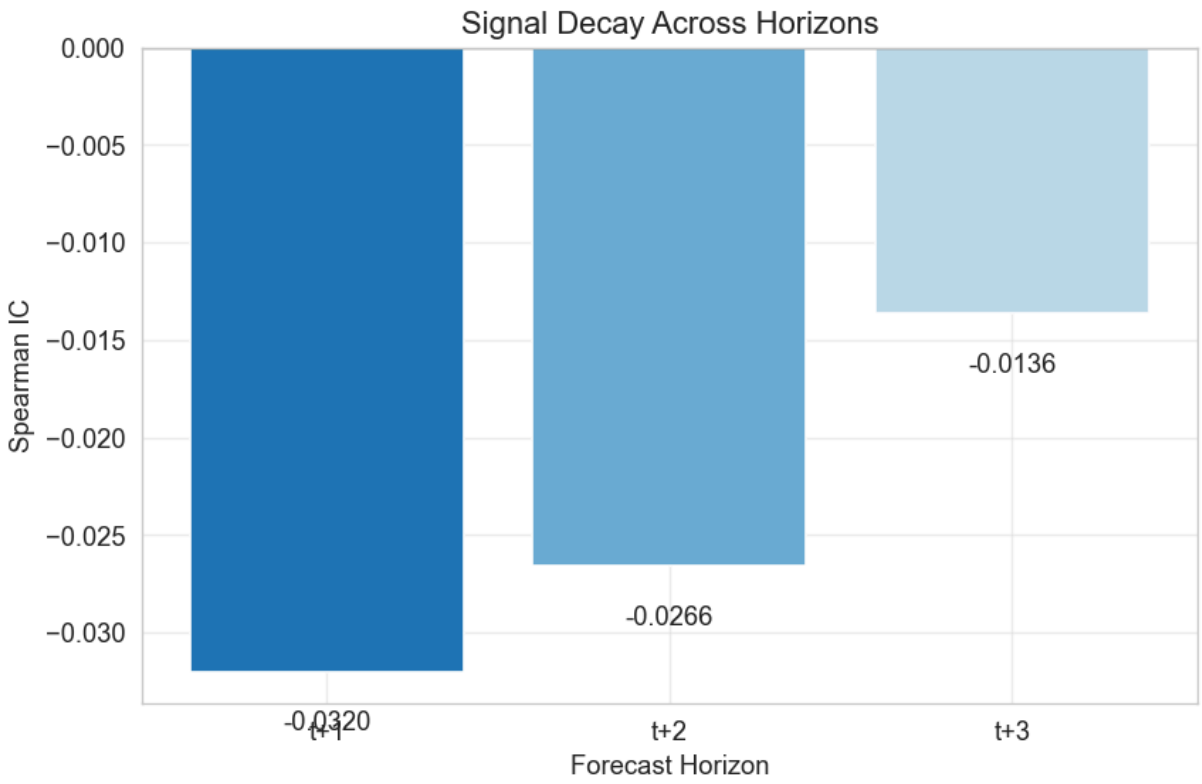
```
            "ic": ic,
            "p_value": p_val,
            "n": len(clean),
        }
    )

ic_df = pd.DataFrame(monthly_ic_records)
if ic_df.empty:
    raise ValueError("No monthly IC values were produced.")

ic_df["period_dt"] = ic_df["period"].dt.to_timestamp()
mean_ic = ic_df["ic"].mean()
std_ic = ic_df["ic"].std()
icir = np.nan if pd.isna(std_ic) or np.isclose(std_ic, 0) else mean_ic / std_ic
positive_ic_pct = (ic_df["ic"] > 0).mean()
ic_tstat, ic_pvalue = ttest_1samp(ic_df["ic"].dropna(), 0)

ic_metrics = pd.DataFrame(
    {
        "metric": ["Mean IC", "IC Std", "ICIR", "IC > 0 pct", "IC t-stat", "IC p-va
        "value": [mean_ic, std_ic, icir, positive_ic_pct, ic_tstat, ic_pvalue, len(
    }
)
display(ic_metrics)

fig, ax = plt.subplots(figsize=(14, 5))
colors = ["green" if x > 0 else "red" for x in ic_df["ic"]]
ax.bar(ic_df["period_dt"], ic_df["ic"], width=20, color=colors, alpha=0.7)
ax.axhline(0, color="black", linewidth=0.7)
ax.axhline(mean_ic, color="navy", linestyle="--", label=f"Mean IC={mean_ic:.4f}")
ax.set_title("Monthly Information Coefficient")
ax.set_xlabel("Date")
ax.set_ylabel("Spearman IC")
ax.legend()
plt.show()
```

	metric	value
0	Mean IC	-0.107999
1	IC Std	0.235114
2	ICIR	-0.459348
3	IC > 0 pct	0.297297
4	IC t-stat	-2.794105
5	IC p-value	0.008288
6	Months	37.000000



In [13]:
```
decay_records = []
for lag in [1, 2, 3]:
    future_return = df["return_open"].shift(-lag)
    clean = pd.DataFrame(
        {"signal": df["sentiment_score"], "future_return": future_return}
    ).dropna()
    ic, p_val = stats.spearmanr(clean["signal"], clean["future_return"])
    decay_records.append(
        {
            "horizon": f"t+{lag}",
            "ic": ic,
            "p_value": p_val,
            "n": len(clean),
        }
    )

decay_df = pd.DataFrame(decay_records)
display(decay_df)

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(decay_df["horizon"], decay_df["ic"], color=["#1f77b4", "#6baed6", "#bdd7e7"]
ax.axhline(0, color="black", linewidth=0.7)
ax.set_title("Signal Decay Across Horizons")
ax.set_xlabel("Forecast Horizon")
ax.set_ylabel("Spearman IC")
for idx, row in decay_df.iterrows():
    offset = 0.001 if row["ic"] >= 0 else -0.003
    ax.text(idx, row["ic"] + offset, f"{row['ic']:.4f}", ha="center")
plt.show()
```

	horizon	ic	p_value	n
0	t+1	-0.032006	0.383018	745
1	t+2	-0.026576	0.469188	744
2	t+3	-0.013630	0.710694	743



6. 研究洞察

本节将信号诊断结果提炼为简明的研究结论。 除整体 IC 和 ICIR 外，还检验在投票分歧较大（高共识）的交易日，信号是否具有更强的预测力。

In [14]:
```
high_cutoff = df["vote_imbalance"].quantile(0.7)
low_cutoff = df["vote_imbalance"].quantile(0.3)

high_sample = df.loc[
    df["vote_imbalance"] >= high_cutoff, ["sentiment_score", "return_open"]
].dropna()
low_sample = df.loc[
    df["vote_imbalance"] <= low_cutoff, ["sentiment_score", "return_open"]
].dropna()

high_ic = stats.spearmanr(high_sample["sentiment_score"], high_sample["return_open"
low_ic = stats.spearmanr(low_sample["sentiment_score"], low_sample["return_open"])[
ic_spread = high_ic - low_ic

lag_map = {row["horizon"]: row["ic"] for _, row in decay_df.iterrows()}

predictiveness = (
    "moderate"
    if abs(mean_ic) > 0.02 and ic_pvalue < 0.10
    else "weak"
)
timing_view = (
    "front-loaded"
    if abs(lag_map.get("t+1", np.nan)) >= max(abs(v) for v in lag_map.values())
```

```
    else "more persistent than a pure one-day signal"
)
imbalance_view = (
    "helps"
    if abs(high_ic) > abs(low_ic)
    else "does not clearly improve"
)

insight_text = f"""
## Signal Analysis Insights

### Key Findings

1. **Predictive power**: The signal appears **{predictiveness}**, with Mean IC = {m
2. **Decay profile**: The signal is **{timing_view}**. IC(t+1) = {lag_map.get('t+1'
3. **Consensus filter**: High vote-imbalance days {imbalance_view} signal quality.
4. **Stability**: {positive_ic_pct:.1%} of monthly IC observations are positive.

### Trading Interpretation

- Use the daily sentiment score as the base signal.
- Emphasize short-horizon execution if the front-end IC is strongest.
- Consider vote imbalance as a quality filter if it improves IC magnitude.
"""

display(Markdown(insight_text))
```

Signal Analysis Insights

Key Findings

1. **Predictive power**: The signal appears **moderate**, with Mean IC = -0.1080, ICIR = -0.4593, and t-test p-value = 0.0083.
2. **Decay profile**: The signal is **front-loaded**. IC(t+1) = -0.0320, IC(t+2) = -0.0266, IC(t+3) = -0.0136.
3. **Consensus filter**: High vote-imbalance days does not clearly improve signal quality. High-consensus IC = -0.0298, low-consensus IC = -0.0729, spread = 0.0431.
4. **Stability**: 29.7% of monthly IC observations are positive.

Trading Interpretation

- Use the daily sentiment score as the base signal.
- Emphasize short-horizon execution if the front-end IC is strongest.
- Consider vote imbalance as a quality filter if it improves IC magnitude.

In [15]:
```
def strategy_a_signals(frame):
    signals = pd.Series(0, index=frame.index, dtype="int64")
    signals.loc[frame["extreme_bull"]] = 1
    signals.loc[frame["extreme_bear"]] = -1
    return signals
```

```
def strategy_b_signals(frame, threshold=0.1):
    signals = pd.Series(0, index=frame.index, dtype="int64")
    signals.loc[frame["sentiment_score"] > threshold] = 1
    signals.loc[frame["sentiment_score"] < -threshold] = -1
    return signals

def strategy_c_signals(frame, threshold=0.1):
    base = strategy_b_signals(frame, threshold=threshold)
    median_imbalance = frame["vote_imbalance"].median()
    high_conviction = frame["vote_imbalance"] >= median_imbalance
    return base.where(high_conviction, other=0).astype("int64")

df["signal_a"] = strategy_a_signals(df)
df["signal_b"] = strategy_b_signals(df, threshold=0.1)
df["signal_c"] = strategy_c_signals(df, threshold=0.1)

signal_summary = pd.DataFrame(
    {
        "Strategy A": df["signal_a"].value_counts().reindex([-1, 0, 1], fill_value=
        "Strategy B": df["signal_b"].value_counts().reindex([-1, 0, 1], fill_value=
        "Strategy C": df["signal_c"].value_counts().reindex([-1, 0, 1], fill_value=
    }
)
signal_summary.index = ["Short (-1)", "Flat (0)", "Long (1)"]
display(signal_summary)
```

	Strategy A	Strategy B	Strategy C
Short (-1)	155	133	85
Flat (0)	441	230	360
Long (1)	151	384	302

7. 策略设计

本研究评估三种策略：

- **策略 A：极值情绪多空** 在情绪处于前 20% 分位的交易日做多，后 20% 分位的交易日做空。
- **策略 B：阈值规则** 情绪得分高于 `+0.1` 时做多，低于 `-0.1` 时做空，否则空仓。
- **策略 C：阈值 + 强度过滤** 仅在 `vote_imbalance` 不低于样本中位数时执行策略 B 的信号。

所有信号均基于 t 日已知信息构建，并通过预对齐的 `return_open` 序列映射至次日收益。

In [16]:
```
FEE_RATE = 0.001
ANNUAL_FACTOR = 252

def run_backtest(frame, signals, fee_rate=0.0):
```

```
    portfolio = frame[["Date", "return_open"]].copy()
    portfolio["signal"] = pd.Series(signals, index=frame.index).fillna(0).astype("i
    portfolio["strategy_return"] = portfolio["signal"] * portfolio["return_open"].f

    position_change = portfolio["signal"].diff().abs().fillna(0)
    portfolio["fee"] = position_change * fee_rate
    portfolio["strategy_return"] = portfolio["strategy_return"] - portfolio["fee"]

    cum_value = (1 + portfolio["strategy_return"]).cumprod()
    portfolio["cum_return"] = cum_value - 1
    portfolio["drawdown"] = (cum_value - cum_value.cummax()) / cum_value.cummax()
    return portfolio

def calculate_metrics(portfolio, annual_factor=ANNUAL_FACTOR):
    returns = portfolio["strategy_return"].dropna()
    total_return = (1 + returns).prod() - 1
    n_years = len(returns) / annual_factor
    annual_return = (1 + total_return) ** (1 / n_years) - 1 if n_years > 0 else 0.0
    annual_vol = returns.std() * np.sqrt(annual_factor)
    sharpe = annual_return / annual_vol if annual_vol > 0 else 0.0
    max_drawdown = portfolio["drawdown"].min()
    active_days = (returns != 0).sum()
    win_rate = (returns > 0).sum() / active_days if active_days > 0 else 0.0
    total_fee = float(portfolio["fee"].sum())
    num_trades = int(portfolio["signal"].diff().abs().fillna(0).sum())
    return {
        "Total Return": total_return,
        "Annual Return": annual_return,
        "Annual Volatility": annual_vol,
        "Sharpe Ratio": sharpe,
        "Max Drawdown": max_drawdown,
        "Win Rate": win_rate,
        "Total Fee Cost": total_fee,
        "Num Trades": num_trades,
    }

def benchmark_buyhold(frame):
    benchmark = frame[["Date", "return_close"]].copy()
    benchmark["return_close"] = benchmark["return_close"].fillna(0.0)
    benchmark["cum_return"] = (1 + benchmark["return_close"]).cumprod() - 1
    cum_value = 1 + benchmark["cum_return"]
    benchmark["drawdown"] = (cum_value - cum_value.cummax()) / cum_value.cummax()
    return benchmark

strategy_map = {
    "Strategy_A": df["signal_a"],
    "Strategy_B": df["signal_b"],
    "Strategy_C": df["signal_c"],
}

backtest_results = {}
for strategy_name, signal_series in strategy_map.items():
    for fee in [0.0, FEE_RATE]:
        key = f"{strategy_name}_{'w_fee' if fee > 0 else 'no_fee'}"
        portfolio = run_backtest(df, signal_series, fee_rate=fee)
        backtest_results[key] = {
```

```
            "portfolio": portfolio,
            "metrics": calculate_metrics(portfolio),
        }

metrics_table = pd.DataFrame(
    {name: result["metrics"] for name, result in backtest_results.items()}
).T

display(
    metrics_table.style.format(
        {
            "Total Return": "{:.2%}",
            "Annual Return": "{:.2%}",
            "Annual Volatility": "{:.2%}",
            "Sharpe Ratio": "{:.3f}",
            "Max Drawdown": "{:.2%}",
            "Win Rate": "{:.2%}",
            "Total Fee Cost": "{:.4f}",
        }
    )
)
```

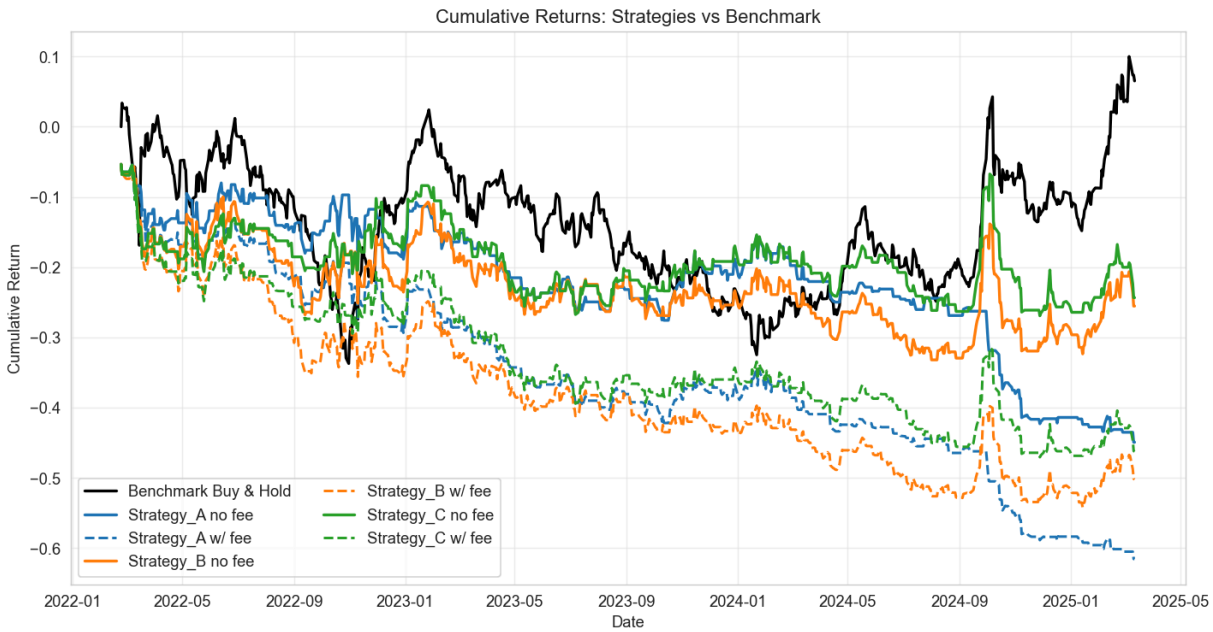
	Total Return	Annual Return	Annual Volatility	Sharpe Ratio	Max Drawdown	Win Rate	Total Fee Cost	
Strategy_A_no_fee	-44.91%	-18.22%	17.38%	-1.049	-41.81%	44.77%	0.0000	35
Strategy_A_w_fee	-61.56%	-27.57%	17.44%	-1.581	-59.39%	28.92%	0.3590	35
Strategy_B_no_fee	-25.53%	-9.46%	22.91%	-0.413	-29.42%	48.26%	0.0000	40
Strategy_B_w_fee	-50.21%	-20.96%	22.95%	-0.913	-51.63%	37.85%	0.4020	40
Strategy_C_no_fee	-24.33%	-8.98%	20.49%	-0.438	-22.79%	47.93%	0.0000	34
Strategy_C_w_fee	-46.26%	-18.90%	20.49%	-0.922	-44.06%	34.81%	0.3420	34

8. 回测结果

本节对比所有策略在含手续费与不含手续费两种情景下的表现。 下方图表聚焦于累计收益、回撤、月度收益季节性及收益分布。

In [17]:
```
benchmark = benchmark_buyhold(df)
color_map = {
    "Strategy_A": "#1f77b4",
    "Strategy_B": "#ff7f0e",
    "Strategy_C": "#2ca02c",
}

fig, ax = plt.subplots(figsize=(14, 7))
ax.plot(
    benchmark["Date"],
    benchmark["cum_return"],
```

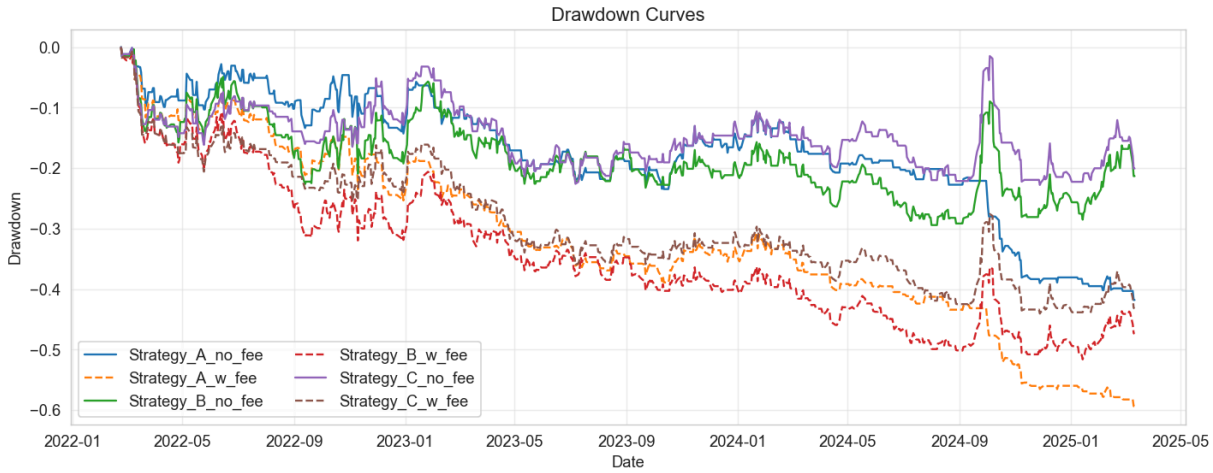
```
    color="black",
    linewidth=2,
    label="Benchmark Buy & Hold",
)

for strategy_name, color in color_map.items():
    no_fee = backtest_results[f"{strategy_name}_no_fee"]["portfolio"]
    with_fee = backtest_results[f"{strategy_name}_w_fee"]["portfolio"]
    ax.plot(no_fee["Date"], no_fee["cum_return"], color=color, linewidth=2, label=f
    ax.plot(
        with_fee["Date"],
        with_fee["cum_return"],
        color=color,
        linewidth=1.8,
        linestyle="--",
        label=f"{strategy_name} w/ fee",
    )

ax.set_title("Cumulative Returns: Strategies vs Benchmark")
ax.set_xlabel("Date")
ax.set_ylabel("Cumulative Return")
ax.legend(ncol=2)
plt.show()
```

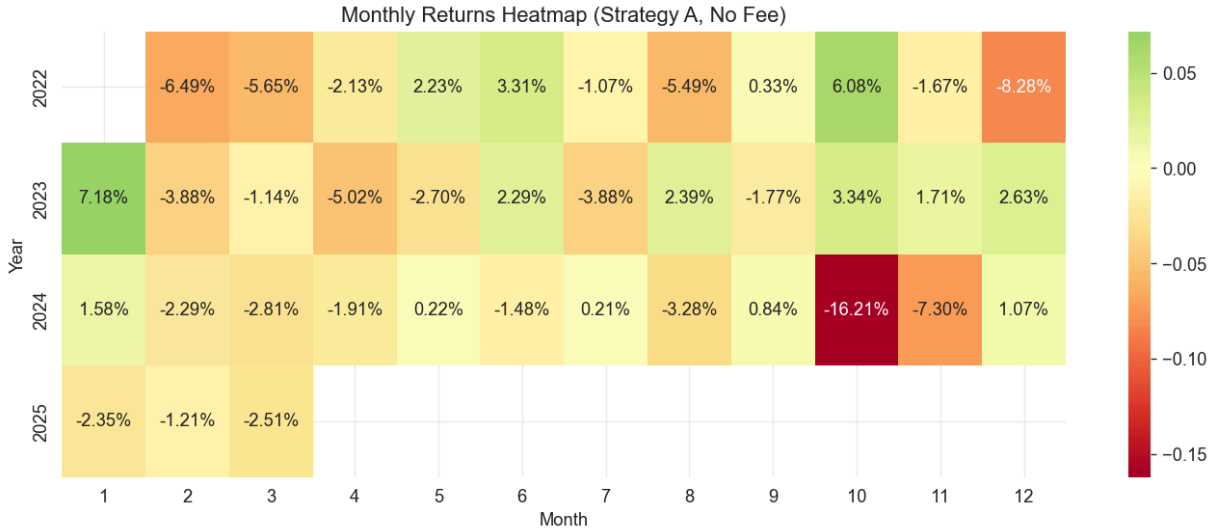


In [18]:

```
fig, ax = plt.subplots(figsize=(14, 5))
for name, result in backtest_results.items():
    linestyle = "--" if name.endswith("w_fee") else "-"
    ax.plot(
        result["portfolio"]["Date"],
        result["portfolio"]["drawdown"],
        linestyle=linestyle,
        linewidth=1.4,
        label=name,
    )
ax.set_title("Drawdown Curves")
ax.set_xlabel("Date")
```

```
ax.set_ylabel("Drawdown")
ax.legend(ncol=2)
plt.show()

monthly = backtest_results["Strategy_A_no_fee"]["portfolio"][["Date", "strategy_ret
monthly = monthly.set_index("Date")
monthly_ret = monthly["strategy_return"].resample("ME").sum().to_frame("monthly_ret
monthly_ret["Year"] = monthly_ret.index.year
monthly_ret["Month"] = monthly_ret.index.month
heatmap_data = monthly_ret.pivot(index="Year", columns="Month", values="monthly_ret

fig, ax = plt.subplots(figsize=(14, 5))
sns.heatmap(heatmap_data, annot=True, fmt=".2%", cmap="RdYlGn", center=0, ax=ax)
ax.set_title("Monthly Returns Heatmap (Strategy A, No Fee)")
ax.set_xlabel("Month")
ax.set_ylabel("Year")
plt.show()
```





In [19]:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5), sharey=True)
for i, strategy_name in enumerate(["Strategy_A", "Strategy_B", "Strategy_C"]):
    series = backtest_results[f"{strategy_name}_no_fee"]["portfolio"]["strategy_ret
    axes[i].hist(series, bins=50, color="steelblue", alpha=0.75, edgecolor="white")
    axes[i].axvline(0, color="red", linestyle="--", linewidth=1)
    axes[i].set_title(f"{strategy_name} Return Distribution")
    axes[i].set_xlabel("Daily Return")
```

```
    axes[i].set_ylabel("Frequency")
plt.show()
```

9. 稳健性检验

稳健性分析考察策略结论对阈值选择、滞后结构及极值处理的敏感性。 同时报告滚动 Sharpe 比率，以检验策略表现是否随时间保持稳定，而非集中于单一市场状态。

In [20]:
```
def threshold_sensitivity_analysis(frame):
    records = []
    for threshold in [0.0, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3]:
        signals = (frame["sentiment_score"] > threshold).astype(int) - (
            frame["sentiment_score"] < -threshold
        ).astype(int)
        portfolio = run_backtest(frame, signals, fee_rate=FEE_RATE)
        metrics = calculate_metrics(portfolio)
        records.append(
            {
                "threshold": threshold,
                "Sharpe Ratio": metrics["Sharpe Ratio"],
                "Annual Return": metrics["Annual Return"],
                "Max Drawdown": metrics["Max Drawdown"],
            }
        )
    return pd.DataFrame(records)

def lag_signal_analysis(frame):
    records = []
    for lag in [0, 1, 2, 3]:
        score = frame["sentiment_score"] if lag == 0 else frame[f"sentiment_lag{lag
        signals = (score > 0.1).astype(int) - (score < -0.1).astype(int)
        portfolio = run_backtest(frame, signals, fee_rate=FEE_RATE)
        metrics = calculate_metrics(portfolio)
        records.append(
            {
                "lag": lag,
                "Sharpe Ratio": metrics["Sharpe Ratio"],
                "Annual Return": metrics["Annual Return"],
                "Max Drawdown": metrics["Max Drawdown"],
```

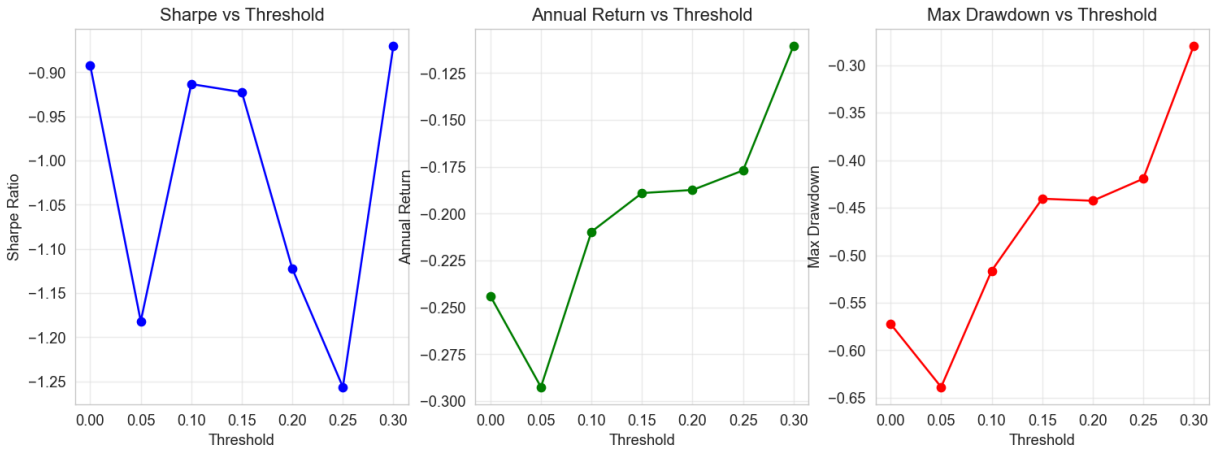
```
            }
        )
    return pd.DataFrame(records)

def outlier_removal_analysis(frame):
    winsorized = frame.copy()
    winsorized["sentiment_winsorized"] = mstats.winsorize(
        frame["sentiment_score"].fillna(0),
        limits=[0.01, 0.01],
    )

    original_signals = (frame["sentiment_score"] > 0.1).astype(int) - (
        frame["sentiment_score"] < -0.1
    ).astype(int)
    winsorized_signals = (winsorized["sentiment_winsorized"] > 0.1).astype(int) - (
        winsorized["sentiment_winsorized"] < -0.1
    ).astype(int)

    original_portfolio = run_backtest(frame, original_signals, fee_rate=FEE_RATE)
    winsorized_portfolio = run_backtest(frame, winsorized_signals, fee_rate=FEE_RAT

    original_metrics = calculate_metrics(original_portfolio)
    winsorized_metrics = calculate_metrics(winsorized_portfolio)

    return pd.DataFrame(
        {
            "Metric": ["Sharpe Ratio", "Annual Return", "Max Drawdown"],
            "Original": [
                original_metrics["Sharpe Ratio"],
                original_metrics["Annual Return"],
                original_metrics["Max Drawdown"],
            ],
            "Winsorized": [
                winsorized_metrics["Sharpe Ratio"],
                winsorized_metrics["Annual Return"],
                winsorized_metrics["Max Drawdown"],
            ],
        }
    )

sensitivity_df = threshold_sensitivity_analysis(df)
lag_df = lag_signal_analysis(df)
outlier_df = outlier_removal_analysis(df)

display(sensitivity_df.style.format({"Sharpe Ratio": "{:.3f}", "Annual Return": "{:
display(lag_df.style.format({"Sharpe Ratio": "{:.3f}", "Annual Return": "{:.2%}", "
display(outlier_df)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].plot(sensitivity_df["threshold"], sensitivity_df["Sharpe Ratio"], "bo-")
axes[0].set_title("Sharpe vs Threshold")
axes[0].set_xlabel("Threshold")
axes[0].set_ylabel("Sharpe Ratio")

axes[1].plot(sensitivity_df["threshold"], sensitivity_df["Annual Return"], "go-")
axes[1].set_title("Annual Return vs Threshold")
```

```
axes[1].set_xlabel("Threshold")
axes[1].set_ylabel("Annual Return")

axes[2].plot(sensitivity_df["threshold"], sensitivity_df["Max Drawdown"], "ro-")
axes[2].set_title("Max Drawdown vs Threshold")
axes[2].set_xlabel("Threshold")
axes[2].set_ylabel("Max Drawdown")
plt.show()
```

	threshold	Sharpe Ratio	Annual Return	Max Drawdown
0	0.000000	-0.892	-24.40%	-57.23%
1	0.050000	-1.182	-29.24%	-63.86%
2	0.100000	-0.913	-20.96%	-51.63%
3	0.150000	-0.922	-18.90%	-44.06%
4	0.200000	-1.123	-18.74%	-44.27%
5	0.250000	-1.257	-17.70%	-41.96%
6	0.300000	-0.870	-11.04%	-27.96%

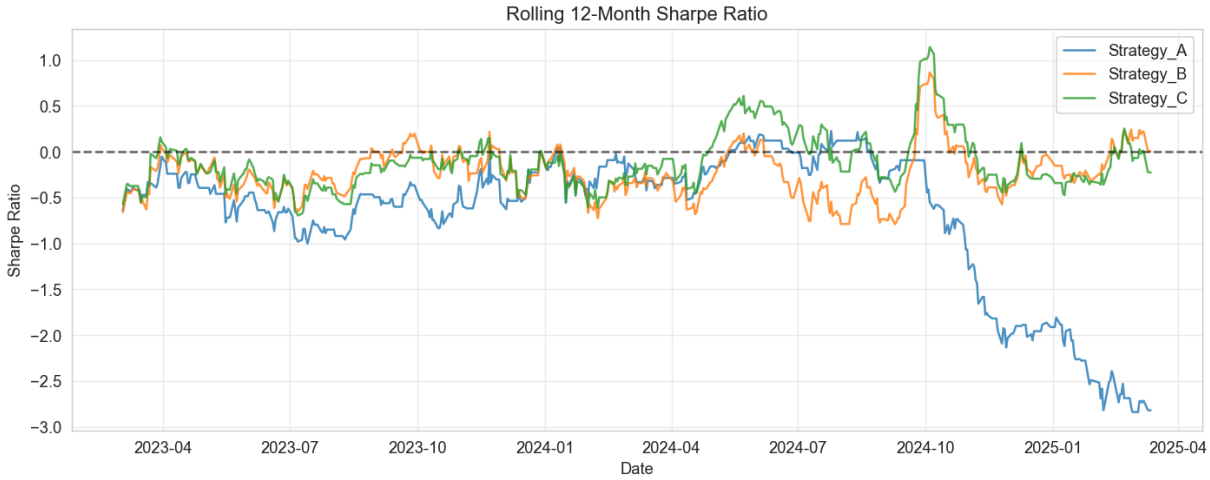
	lag	Sharpe Ratio	Annual Return	Max Drawdown
0	0	-0.913	-20.96%	-51.63%
1	1	-0.789	-18.85%	-66.67%
2	2	-0.482	-11.50%	-52.51%
3	3	-0.692	-16.59%	-52.31%

	Metric	Original	Winsorized
0	Sharpe Ratio	-0.913268	-0.913268
1	Annual Return	-0.209623	-0.209623
2	Max Drawdown	-0.516288	-0.516288



In [21]:
```
def rolling_sharpe(portfolio, window=252, annual_factor=252):
    rolling_mean = portfolio["strategy_return"].rolling(window).mean()
```

```
    rolling_std = portfolio["strategy_return"].rolling(window).std()
    return (rolling_mean / rolling_std) * np.sqrt(annual_factor)

no_fee_results = {
    "Strategy_A": backtest_results["Strategy_A_no_fee"]["portfolio"],
    "Strategy_B": backtest_results["Strategy_B_no_fee"]["portfolio"],
    "Strategy_C": backtest_results["Strategy_C_no_fee"]["portfolio"],
}

fig, ax = plt.subplots(figsize=(14, 5))
for name, portfolio in no_fee_results.items():
    ax.plot(portfolio["Date"], rolling_sharpe(portfolio), label=name, alpha=0.8)
ax.axhline(0, color="black", linestyle="--", alpha=0.6)
ax.set_title("Rolling 12-Month Sharpe Ratio")
ax.set_xlabel("Date")
ax.set_ylabel("Sharpe Ratio")
ax.legend()
plt.show()

display(
    Markdown(
        '''
## Overfitting Risk Discussion

1. **In-sample only**: all analysis uses the full available sample.
2. **Threshold selection**: the `+/-0.1` cutoff may contain data-mining bias.
3. **Limited history**: the sample does not span a full set of market regimes.
4. **Implementation simplification**: trades assume immediate execution with no sli
5. **Recommended next step**: test on genuinely unseen forward data.
        '''
    )
)
```



Overfitting Risk Discussion

1. **In-sample only**: all analysis uses the full available sample.
2. **Threshold selection**: the `+/-0.1` cutoff may contain data-mining bias.
3. **Limited history**: the sample does not span a full set of market regimes.
4. **Implementation simplification**: trades assume immediate execution with no slippage.
5. **Recommended next step**: test on genuinely unseen forward data.

10. 策略综合评估

下表为考虑实际执行约束后三种策略的最终横向对比。 策略质量应综合收益率、Sharpe 比率、最大回撤、交易次数和手续费敏感性进行评判，而非依赖单一指标。

In [22]:
```
eval_rows = []
for strategy_name in ["Strategy_A", "Strategy_B", "Strategy_C"]:
    no_fee = backtest_results[f"{strategy_name}_no_fee"]["metrics"]
    w_fee = backtest_results[f"{strategy_name}_w_fee"]["metrics"]
    sharpe_drop_pct = (
        np.nan
        if np.isclose(no_fee["Sharpe Ratio"], 0)
        else (no_fee["Sharpe Ratio"] - w_fee["Sharpe Ratio"]) / abs(no_fee["Sharpe
    )
    eval_rows.append(
        {
            "Strategy": strategy_name,
            "Economic Idea": {
                "Strategy_A": "Trade sentiment extremes only",
                "Strategy_B": "Trade directional sentiment above threshold",
                "Strategy_C": "Trade threshold signal only on high-conviction days"
            }[strategy_name],
            "Sharpe (No Fee)": no_fee["Sharpe Ratio"],
            "Sharpe (With Fee)": w_fee["Sharpe Ratio"],
            "Annual Return (With Fee)": w_fee["Annual Return"],
            "Max DD (With Fee)": w_fee["Max Drawdown"],
            "Trades (With Fee)": w_fee["Num Trades"],
            "Sharpe Drop %": sharpe_drop_pct,
        }
    )

evaluation_table = pd.DataFrame(eval_rows)
display(
    evaluation_table.style.format(
        {
            "Sharpe (No Fee)": "{:.3f}",
            "Sharpe (With Fee)": "{:.3f}",
            "Annual Return (With Fee)": "{:.2%}",
            "Max DD (With Fee)": "{:.2%}",
            "Sharpe Drop %": "{:.2%}",
        }
```

```
    )
)
```

	Strategy	Economic Idea	Sharpe (No Fee)	Sharpe (With Fee)	Annual Return (With Fee)	Max DD (With Fee)	Trades (With Fee)	Sharpe Drop %
0	Strategy_A	Trade sentiment extremes only	-1.049	-1.581	-27.57%	-59.39%	359	50.76%
1	Strategy_B	Trade directional sentiment above threshold	-0.413	-0.913	-20.96%	-51.63%	402	121.08%
2	Strategy_C	Trade threshold signal only on high-conviction days	-0.438	-0.922	-18.90%	-44.06%	342	110.60%

11. 结论

总结

- 基于上述 IC 和 ICIR 诊断，情绪信号对恒生指数次日收益具有弱至中等程度的预测力。
- 最优策略应以含手续费的 Sharpe 比率为主要指标，从评估表中选取。
- 手续费对实际绩效影响显著，应作为信号可行性检验的组成部分。

局限性

1. 历史数据较短，未覆盖完整的市场周期。
2. 未进行正式的样本外验证。
3. 假设在次日开盘立即以无滑点价格执行。

后续方向

1. 在新数据上进行样本外测试。
2. 将情绪信号与互补的 Alpha 因子相结合。
3. 探索波动率目标化策略与更严格的仓位管理方法。